Research Plan: LLaMA Group

Layer 1 – Path A: Grammar-Constrained Decoding (GCD) with XGrammar

XS-BNF / SWPC-BNF + LLM Integration Project

August 26, 2026

1 Objective

Implement and evaluate **Grammar-Constrained Decoding (GCD)** for the LLaMA model family using **XGrammar**, with SWPC-BNF rules as the constraint grammar. The goal is to force LLaMA to generate *syntactically valid* Chinese character structural descriptions at inference time, with **zero modification** to model weights or architecture.

2 Motivation

In baseline tests, LLaMA demonstrated critical weaknesses in Chinese character cognition:

- When prompted *"como escrever agua em Chines"* (how to write "water" in Chinese), LLaMA failed to respond, while Gemma provided a cross-lingual explanation with the correct character "水".
- LLaMA tends to **hallucinate** non-existent character components or illegal structural compositions when asked to describe or generate Chinese characters.

Rather than fine-tuning (which is costly and may not generalize), we will impose **hard structural constraints at the decoding layer** using XGrammar. If LLaMA is forced to sample only tokens that conform to SWPC-BNF production rules, its output becomes structurally valid by construction.

3 Methodology: XGrammar GCD

3.1 Core Principle

XGrammar compiles a context-free grammar (in our case, SWPC-BNF encoded as EBNF) into an efficient **token-level bitmask**. At each decoding step, LLaMA's next-token probabilities are masked: tokens that would violate the grammar receive probability $-\infty$. This guarantees 100% syntactic validity of the generated string with *near-zero overhead*.

3.2 SWPC-BNF → XGrammar Compilation

SWPC-BNF production rules will be translated into XGrammar-compatible EBNF. Example fragment:

```
char        ::= binary | ternary | quaternary | terminal
binary      ::= lr | tb
```

```
lr          ::= "LR(" component "," component ")"
tb          ::= "TB(" component "," component ")"
component   ::= terminal | binary | ternary
terminal    ::= "木" | "目" | "雨" | "日" | "口" | ...
```

The full rule set covers all 11 structural patterns (LR, TB, TMB, LMR, enclosures, etc.) plus the 26×26 primitive component matrix.

4 Experimental Design

4.1 Task 1: Structural Validity (Pass/Fail)

Prompt LLaMA to generate the SWPC-BNF encoding for given characters.

- **Baseline**: Unconstrained decoding. Measure how often LLaMA produces malformed structures (e.g., missing parentheses, illegal nesting, non-existent components).
- **GCD**: XGrammar-constrained decoding. Measure structural validity rate (target: 100%).

4.2 Task 2: Cross-Lingual Character Description

Replicate the "agua" prompt scenario:

- Prompt (PT): *"Descreva a estrutura do caractere chinês para 'água' em formato SWPC-BNF."*
- Expected: A valid structural description ("水" is a primitive, so output should identify it as terminal).
- Compare LLaMA's performance **with** and **without** GCD.

4.3 Task 3: Compositional Generalization ("Finite → Infinite")

Test zero-shot generalization to **unseen** characters:

- Provide SWPC-BNF rules as grammar context.
- Ask LLaMA to generate the structure of a rare / newly composed character whose components are known but whose exact composition is not in training examples.
- Evaluate whether GCD enables correct recursive composition.

5 Evaluation Metrics

Table 1: Evaluation Metrics for XGrammar GCD Experiments

Metric	**Description**	**Target**
Structural Validity	% outputs parseable by SWPC-BNF parser	$\geq 98\%$
Component Accuracy	Correct primitive components identified	$\geq 90\%$
Nesting Correctness	Correct recursive depth and operators (TB vs TMB, LR vs LMR)	$\geq 85\%$
Cross-lingual Success	Correct response to PT/EN prompts about Chinese characters	$\geq 80\%$
Latency Overhead	ms increase per token vs. unconstrained decoding	$< 5\%$

Table 2: Layer 1 – Path A: XGrammar GCD Technical Specification

Property	Configuration	Notes
Model	LLaMA-3.1-8B-Instruct (or LLaMA-3.2-3B if compute-limited)	Use HuggingFace `transformers` + `xgrammar`
Constraint Engine	XGrammar $\geq$ v0.1.5	Supports vLLM, SGLang, and native HF integration
Grammar Input	SWPC-BNF EBNF file (`swpc_bnf.ebnf`)	Compiled from 26×26 component matrix + 11 structural operators
Decoding Mode	Greedy or nucleus sampling with grammar mask	Mask applied at *every* forward pass
Model Modification	**None** (zero parameter updates)	Weights frozen; only logits are masked
Training Data	Not required for Layer 1	If time permits, optional few-shot examples in prompt
Output Format	BNF bracket expression: `TB(雨, LR(木, 目))`	Directly parseable by CJK-SQL

6 Layer 1: Decoding Constraint Architecture

7 Deliverables

1. **XGrammar EBNF file**: Complete SWPC-BNF grammar specification for XGrammar compiler.
2. **Inference script**: Python script integrating `transformers` + `xgrammar` for constrained generation.
3. **Evaluation report**: Baseline vs. GCD comparison across Tasks 1–3 with metrics from Table 1.
4. **Demo notebook**: A live Colab/Jupyter notebook showing LLaMA generating valid SWPC-BNF structures for 10 test characters.

8 Timeline (4 Weeks)

Week	Milestone	Owner
1	Install XGrammar; compile SWPC-BNF core rules (LR, TB, TMB, terminals)	All
2	Implement GCD pipeline; run Task 1 (validity) on 50 test characters	All
3	Run Task 2 (cross-lingual) and Task 3 (generalization); collect metrics	All
4	Write report; prepare demo; present findings to class	All

9 Key Advantages of This Approach

- **Zero training cost**: No GPUs needed for fine-tuning.

- **100% structural validity guaranteed**: By construction, not by luck.
- **Interpretable**: Every output is a parse tree; errors are localized to the grammar rule level.
- **Foundation for deeper layers**: Once Layer 1 proves that LLaMA *can* speak BNF, Layer 2 (structured tokenization) and Layer 3 (LoRA/MoE) become natural next steps.

10 Expected Insight

If LLaMA with XGrammar GCD successfully generates valid SWPC-BNF encodings while the unconstrained baseline fails, we demonstrate a foundational principle of the XS-BNF framework: *structural knowledge need not be learned from data; it can be imposed as a formal prior.* This is the "finite rules generating infinite valid structures" philosophy in action.

Questions? Contact the project coordinator. Good luck!